

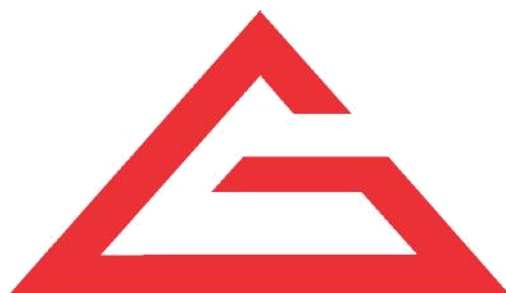


---

# FLUTTER-02

---

## LAB ASSIGNMENTS



**G-TEC EDUCATION**

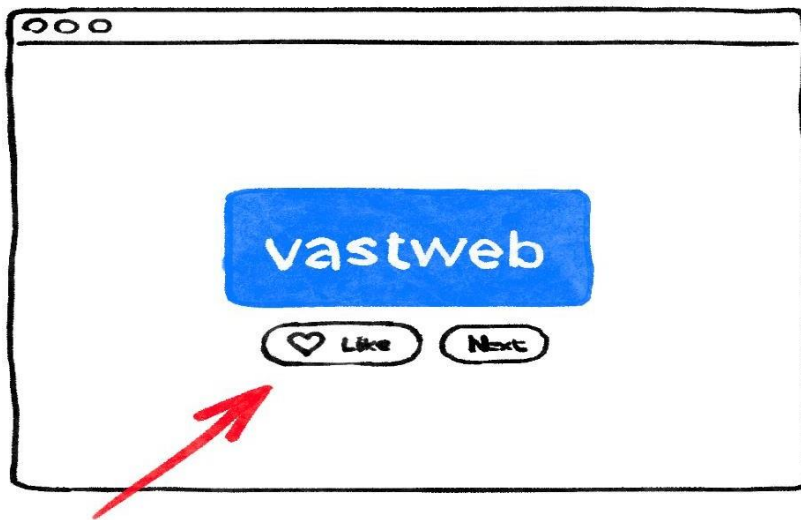
— G-TEC Group of Institutions —

[www.gteceducation.com](http://www.gteceducation.com)

Admin Office House of G-TEC, Calicut-02., India. | Corp. Office Peace Centre, Singapore – 228149

## Add Functionality

The app works, and occasionally even provides interesting word pairs. But whenever the user clicks **Next**, each word pair disappears forever. It would be better to have a way of "remembering" the best suggestions: such as a 'Like' button.



## Add the business logic

Scroll to `MyAppState` and add the following code:

### [lib/main.dart](#)

```
// ...
```

```
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
  
  void getNext() {  
    current = WordPair.random();  
    notifyListeners();  
  }  
}
```

```
}  
  
// ↓ Add the code below.  
var favorites = <WordPair>[];  
  
void toggleFavorite() {  
    if (favorites.contains(current)) {  
        favorites.remove(current);  
    } else {  
        favorites.add(current);  
    }  
    notifyListeners();  
}  
}  
  
// ...
```

Examine the changes:

- You added a new property to `MyAppState` called `favourites`. This property is initialized with an empty list: `[]`.
- You also specified that the list can only ever contain word pairs: `<WordPair>[]`, using [generics](#). This helps make your app more robust—Dart enforces to even *un* your app if you try to add anything other than `WordPair` to it. In turn, you can use the `favorites` list knowing that there can never be any unwanted objects (like `null`) hiding in there.

**Note:** Dart has collection types other than `List` (expressed with `[]`). You could argue that a `Set` (expressed with `{}`) would make more sense for a collection of favourites. To make this codelab as straightforward as possible, we're sticking with a list. But if you want, you can use a `Set` instead. The code wouldn't change much.

- You also added a new method, `toggleFavorite()`, which either removes the current word pair from the list of favourites (if it's already there), adds it. In either case, the code calls `notifyListeners()` afterwards.

## Add the button

With the "business logic" out of the way, it's time to work on the user interface again. Placing the 'Like' button to the left of the 'Next' button requires a `Row`. The `Row` widget is the horizontal equivalent of `Column`, which you saw earlier.

First, wrap the existing button in a `Row`. Go to `MyHomePage`'s `build()` method, put your cursor on the `ElevatedButton`, call up the **Refactor** menu with `Ctrl+Cmd+.`, and select **Wrap with Row**.

When you save, you'll notice that `Row` acts similarly to `Column`—by default, it lumps its children to the left. (`Column` lumps its children to the top.) To fix this, you could use the same approach as before, but with `mainAxisAlignment`. However, for layout purposes, use `mainAxisSize`. This tells `Row` not to take all available horizontal space.

Make the following change:

```
lib/main.dart
```

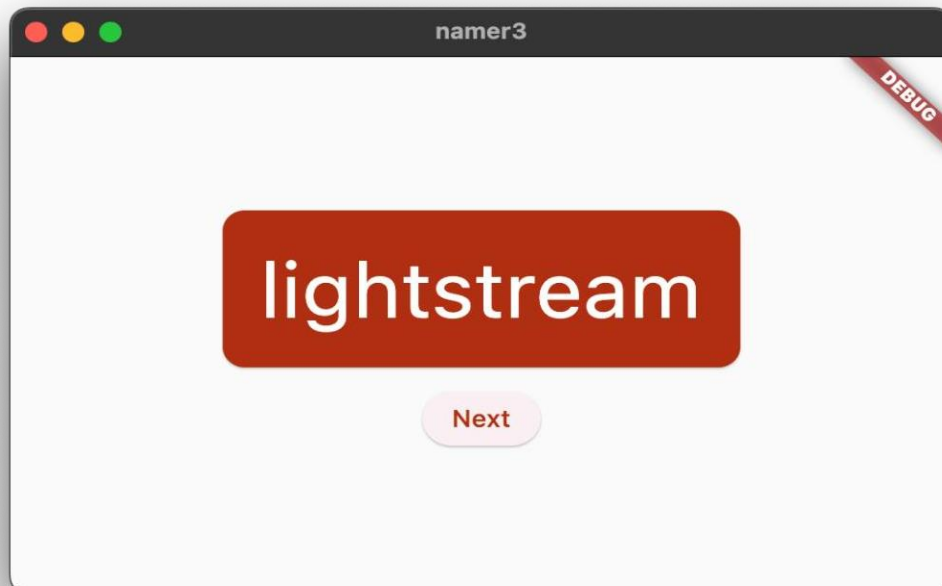
```
// ...
```

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {
```

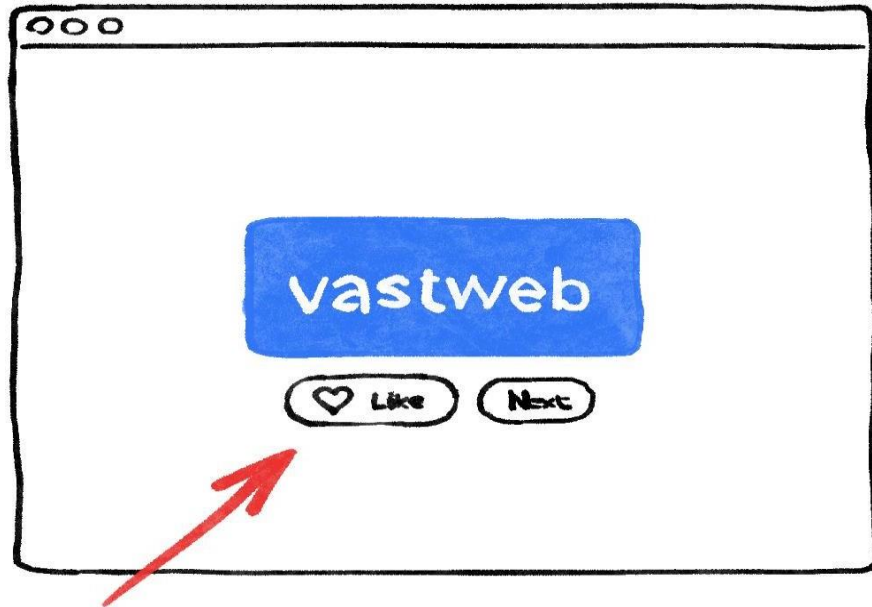
```
var appState = context.watch<MyAppState>();  
var pair = appState.current;  
  
return Scaffold(  
  body: Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: [  
        BigCard(pair: pair),  
        SizedBox(height: 10),  
        Row(  
          mainAxisAlignment: MainAxisAlignment.min, // ← Add this.  
          children: [  
            ElevatedButton(  
              onPressed: () {  
                appState.getNext();  
              },  
              child: Text('Next'),  
            ),  
          ],  
        ),  
      ],  
    ),  
  ),  
);
```

```
};  
}  
}  
// ...
```

the UI is back to where it was before.



Next, add the **Like** button and connect it to `toggleFavorite()`. For a challenge, first try to do this by yourself, without looking at the code block below.



It's okay if you don't do it exactly the same way as it's done below. In fact, don't worry about the heart icon unless you early want a major challenge.

It's also completely okay to fail—this is your first hour with Flutter, after all.

↓ **SPOILERS AHEAD** ↓

Here's one way to add the second button to `MyHomePage`. This time, use the `ElevatedButton.icon()` constructor to create a button with an icon. And at the top of the `build` method, choose the appropriate icon depending on whether the current word pair is already in favourites. Also, note the use of `SizeBox` again, to keep the two buttons a bit apart.





**mainAxisSize:**

**MainAxisSize.min,children: [**

**// ↓ And this.**

**ElevatedButton.icon(**

**onPressed: () {**

**appState.toggleFavorite();**

**},**

**icon:**

**Icon(icon),**

**label:**

**Text('Like'),**

**),**

**SizedBox(width: 10),**

**ElevatedButton(**

**onPressed: () {**

**appState.getNext();**

**},**

**child: Text('Next'),**

**),**

**],**

**),**

**],**

**),**

**),**

**);**

**}**

**}**

// ...